

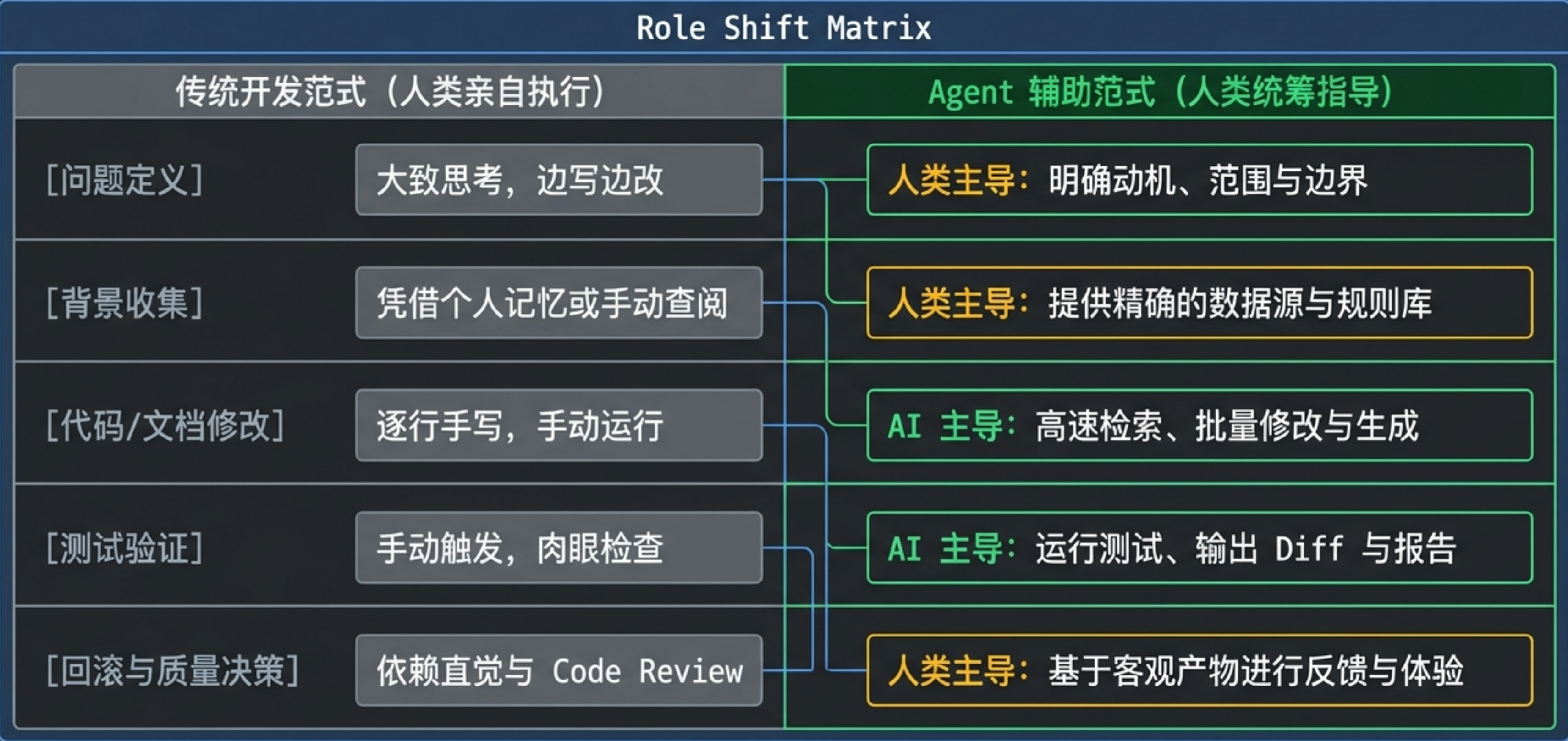
构建可控的 Agent 工程 workflow

从技术演进到 OpenCode 真实系统改造指南

面向具备**软件开发、数据分析与自动化**经验的**工程人员**及**AI 应用构建者**。
这不是一份“如何写 Prompt”的说明书，而是一套重塑人机协作架构的**标准作业程序（SOP）**。

人类核心价值的范式转移：从“执行者”到“导演”

当 AI 具备极强的执行能力后，质量瓶颈不再是“能不能做”，而是“有没有讲清楚”。



人机交接漏斗：在两端施加控制，在中间释放效率

问题定义 (Brief)

人类权重最高。确定剧本、风格、目标、边界和评价标准。

(动机 / 背景 / 规则 / 验收标准)

探索执行 (Plan & Build)

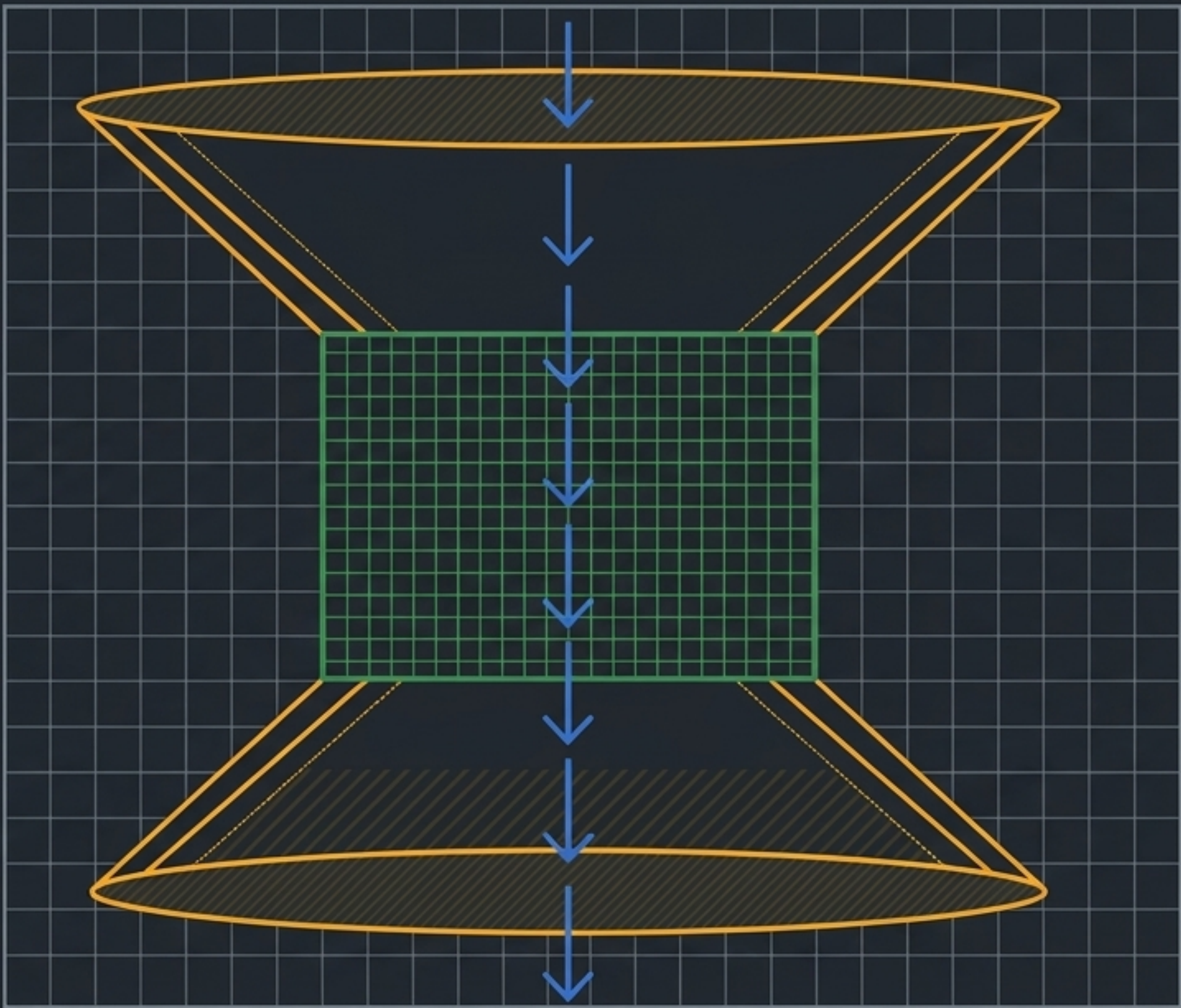
AI 权重最高。充当极强执行力的剧组。

(快速阅读 / 搜索 / 局部修改 / 验证)

验证审查 (Review)

人类权重回归。导演验收成片，决定迭代方向。

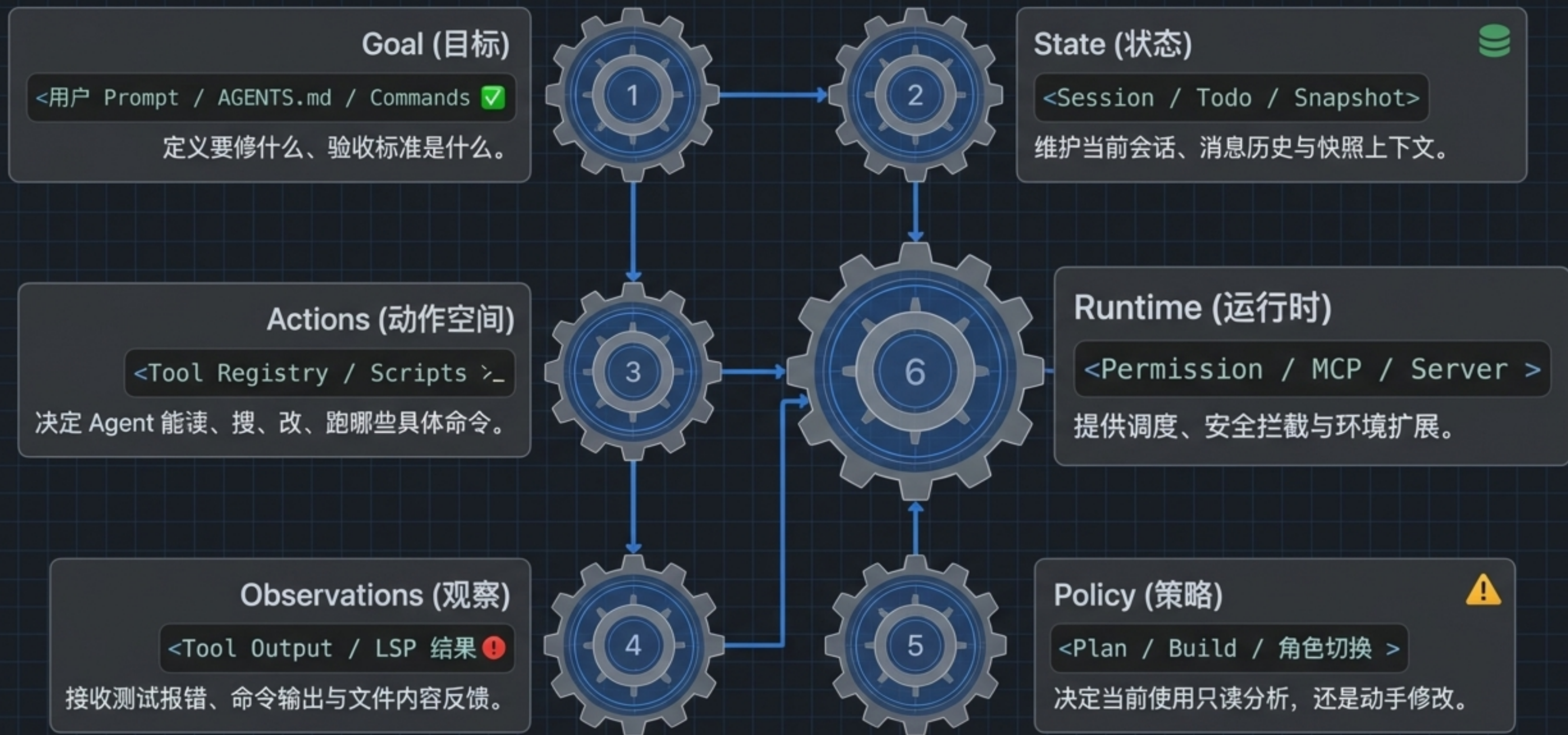
(体验结果 / 检查风险 / 提供反馈)



演进阶梯：从“会思考的模型”到“能安全改代码的运行时代”



引擎解剖：六个槽位驱动的本地 workflow



安全施工边界：权限红绿灯系统

安全不能仅靠 Prompt，必须写进运行时。原则：读操作放宽，写操作阻拦，毁灭性操作默认拒绝。

[终端绿

Allow: 低风险，自动执行]

- read (除 .env 外的普通代码文件)
- git status, git diff, git log
- 短时间测试命令 (pnpm test, lint)

[警戒琥珀

Ask: 中风险，人类确认]

- edit, write, apply_patch (修改类操作)
- bash (执行自定义脚本)

[红底 Deny:

高风险，严格禁止]

- read (.env, .env.* 敏感配置)
- git push, git reset, git clean
- pkill, killall, rm -rf (易误杀 Agent 自身)
- 后台常驻服务 (npm run dev &)

双重保险防线：正确的修改起手式

在真实工程里，不要将 Agent 的自带撤销作为唯一回滚方案。

[前线轻防盾] 副保险：OpenCode /undo

- 定位：轻量级，会话内便利撤销。✅
- 适用：防失误，单次小范围 Tool Call 错误。
- 局限：跨系统或复杂重构易导致上下文脱节。⚠️

[后方防波堤] 主保险：Git Checkpoint

- 定位：工程级，坚不可摧的底层状态保护。✅
- 适用：防事故，多轮修改或执行高危命令前。⚠️
- SOP：修改前先 `git status` -> 工作区干净 -> `git stash push -u -m 'checkpoint before opencode'` 或新建分支。

核心操作规范：先 Plan 分析，再 Build 动手

不要让 Agent 一上来就自由发挥改代码。先让它取证，再让它执行。



PLAN 模式（受限代理 - 智囊团）

核心目标：想清楚。定位入口、分析风险、制定方案。

权限限制：edit 与 write 强制设为 deny。不允许修改。

输出形态：相关文件路径、当前调用链、证据片段、最小修复方案。

人类介入点：审查计划。确认修改范围与测试策略是否合理。



BUILD 模式（开发代理 - 执行队）

核心目标：动手做。按计划执行最小代码修改与验证。

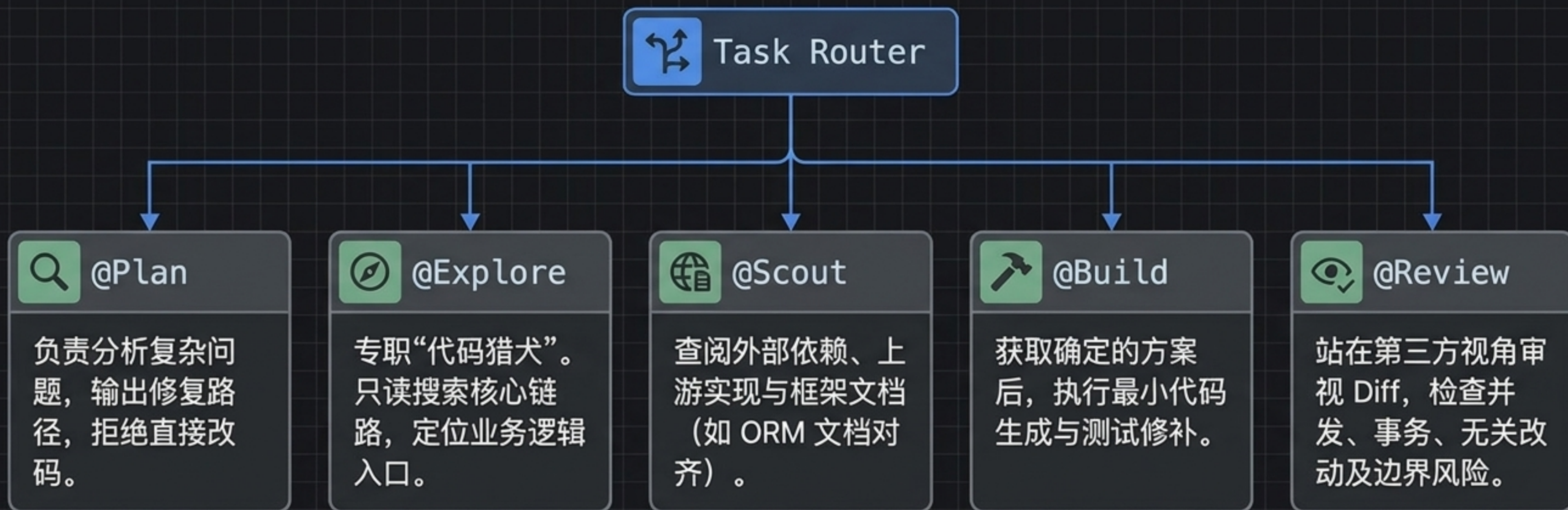
权限限制：拥有受控的文件修改与测试执行权限（仍受全局 Deny 约束）。

输出形态：Diff 摘要、运行最小相关测试的结果。

人类介入点：Code Review。独立验证其实现的正确性。

子代理系统：隔离不同类型的工作空间

多 Agent 的价值是并行取证与角色隔离，而不是并行制造冲突。



反模式： 让多个 Agent 同时在没有 **Checkpoint** 的情况下抢占修改同一批文件。

任务建模：好的输入不是更长，而是信息结构完整

Git Diff Code Review

Console Dark

[反面教材]

> 帮我重构支付模块。

- ✗ 任务过大 ✗ 范围模糊 ✗ 缺乏动机
- ✗ 无验收标准 ✗ 未界定文件边界

[结构化任务卡]

> 先不要改代码。请只读分析支付模块中 `retry`、`charge` 等相关实现。

【动机】 近期出现疑似重复扣款，怀疑与 `token refresh` 或幂等键有关。

【目标】 找出可能发生位置，提出最小修复方案。

【范围】 优先查看

`src/payment`, `tests/payment`。

【约束】 不要修改文件，不要运行改变 DB 状态的命令。 不要修改文件，不要运行改变 DB 状态的。

【输出要求】

1. 涉及调用链
2. 证据片段
3. 补测建议
4. 需人工确认的卡点。

上下文分层架构 (Context Layer Cake)

切忌将所有资料塞入单次对话。将隐性意图外部化为 AI 可读取的系统资产。



资产沉淀：通用 Agent 工程的项目骨架

一次临时对话会随窗口消失，将其沉淀为目录结构，才能打造越用越聪明的系统。

PROJECT EXPLORER

项目根目录

AGENTS.md

(项目说明书：目录约定、测试规范、安全边界)

opencode.json

(运行时网关：文件与命令权限拦截配置)

.opencode/

commands/

(高频入口：如 /analyze, /migrate)

skills/

(固化流程：输入要求、调用顺序、人工 Review 清单)

tools/

(受控动作：封装的内部 API 与脚本)

references/

(对齐依据：背景文档、历史代码样例、数据 Schema)

tests/

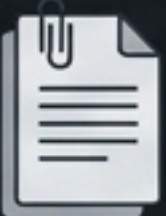
(客观防线：验收脚本，让 AI 有法可依)

logs/

(复盘材料：执行轨迹与人类反馈日志)

数据输入范式：结构化与非结构化文件的任务建模

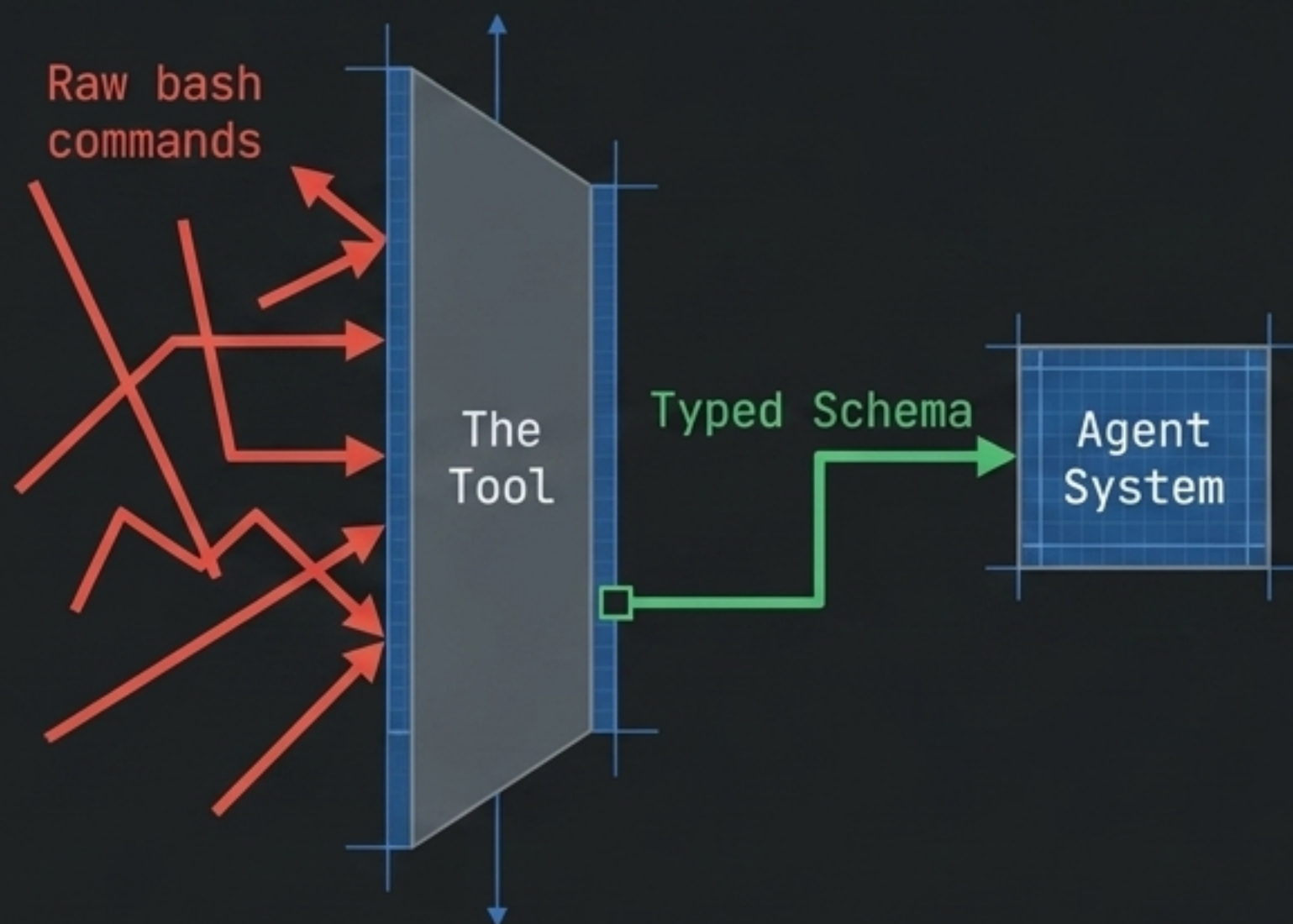
文件处理是高频起点。未经约束的 AI 会产生看似合理但业务语义错误的幻觉。

数据类型	核心处理范式	必须定义的边界与约束
 结构化文件 (CSV, JSON, DB)	基于口径运算	1. 明确主键与分组字段含义。 2. 敲定缺失值/异常值处理规则。 3. 提供验证样例与输出 Schema。
 非结构化文件 (PDF, 网页, 长日志)	先取证，后结论	1. 强制分离：先抽取 Evidence 片段，不许直接生成结论。 2. 关联溯源：保留抽取结果的原始出处。 3. 关键节点设定强制人工 Review。

动作空间收敛：用受控工具替代自由发挥

不要让 Agent 临时拼命令或猜测内网 API。高频动作必须被封装为工程脚本。

API GATEWAY: ACTION FILTERING



高质量 Tool 封装的 6 个要素：

- 1. **输入`Schema`**：参数强类型化，消灭自然语言歧义。
- 2. **输出`Schema`**：返回结构化数据（而非纯文本），便于程序断言与复用。
- 3. **权限沙箱**：仅暴露所需字段，硬编码阻断越权访问。
- 4. **防御性报错**：失败时抛出带有指导意义的`Reason`，拒绝静默失败。
- 5. **审计日志**：记录触发时间、参数与结果，确保执行可追溯。
- 6. **可测试性**：工具自身脱离`Agent`也能通过单元测试。

独立验证体系：没有客观检验，就没有可靠 workflow

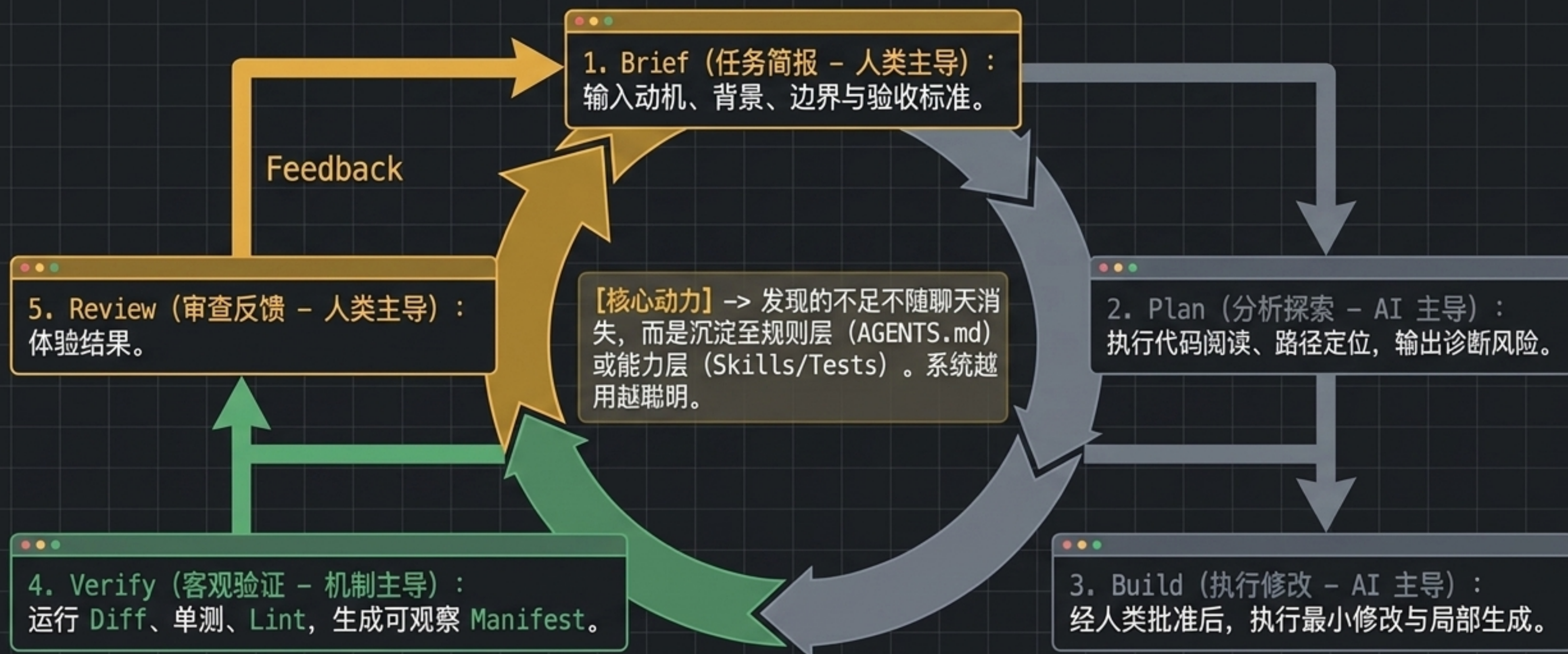
自动验证代替不了业务判断，但能过滤掉 90% 的低级错误。验证必须独立于实现。



交接规范： Agent 完成任务必须提供摘要——根因、修改点、新增测试、失败覆盖项。

五阶段无尽循环 (The 5-Stage Infinite Loop)

真正的 Agent 工程是一台以人类反馈为燃料、持续进化的闭环引擎。



资产合成：何时将对话升级为系统 Skill？

不要让团队反复从零开始解释业务上下文。



落地执行：从低风险场景启动系统改造

[工程纪律口诀]

- 先**规划 (Plan)**，后**执行 (Build)**。
- 先 **Checkpoint (Git)**，后放手修改。
- 人类定方向与反馈，AI 做执行与取证。

[明日起步：三大低风险场景]

1. **逆向工程**：让 Agent 借助探索工具，解释并标注陌生的遗留模块。
2. **防御加固**：让 Agent 为范围明确的核心函数补充极端边界单元测试。
3. **精准修复**：在排查出明确日志堆栈后，让 Agent 修补单一文件内的明确 Bug。

[长效演进目标]

将人类从重复执行中解放，专注于定义问题、判断质量、沉淀全局架构资产 (AGENTS.md, Tests, Skills)。